

Q1. Wrapper Classes and Autoboxing – 5 marks

Problem Statement:

You are developing a student-score processing system where scores are initially stored as primitive integers. To work with Java collections, these values need to be converted into wrapper objects. Java provides autoboxing and unboxing to perform these conversions automatically.

Write a program that reads an integer n , stores it in an Integer wrapper object using autoboxing, and then retrieves the primitive value using unboxing. Print both the wrapper value and the primitive value.

Input:

Each test case contains an integer n , where $-100000 \leq n \leq 100000$.

Output:

Print the value stored in the Integer object and the unboxed integer value on separate lines.

Example:

Input:

25

Output:

25

25

Q2. ArrayList – Dynamic Collection – 5marks

Problem Statement:

You are maintaining a list of marks obtained by students. Since the number of students can change dynamically, an ArrayList is used instead of a fixed-size array.

Write a Java program that reads n student marks, stores them in an ArrayList<Integer>, and prints all the marks in the same order. Then print the highest mark in the list.

Input:

The first line contains an integer n , where $1 \leq n \leq 100$. The second line contains n integers representing student marks, where $0 \leq \text{marks} \leq 100$.

Output:

Print the marks in their original order on the first line. Print the highest mark on the second line.

Example:

Input:

5

78 92 65 88 95

Output:

78 92 65 88 95

95

Q3. Custom Exception – Validating Account Balance -10marks

Problem Statement:

You are developing a banking application where a customer can withdraw money from their account. The application should prevent withdrawals when the requested amount is greater than the available balance. To handle this situation clearly, the application uses a custom exception called `InsufficientBalanceException`.

Write a Java program that reads the account balance and withdrawal amount. Create a custom exception named `InsufficientBalanceException`. If the withdrawal amount is greater than the available balance, throw the custom exception and print "Insufficient Balance". Otherwise, deduct the withdrawal amount from the balance and print the remaining balance. Use `throw`, `throws`, `try`, and `catch` appropriately.

Input:

The first line contains an integer representing the account balance. The second line contains an integer representing the withdrawal amount. Constraints: $0 \leq \text{balance} \leq 1000000$ and $0 \leq \text{withdrawal} \leq 1000000$.

Output:

If the withdrawal is successful, print the remaining balance. If the withdrawal amount exceeds the balance, print "Insufficient Balance".

Example:

Input:

5000

2000

Output:

3000

Q4. Exception Handling – try, catch and finally – 5Marks

Problem Statement:

You are building a calculator application that performs division between two integers. Division by zero causes a runtime error. The program should handle this error using structured exception handling instead of terminating unexpectedly.

Write a Java program that reads two integers `a` and `b` and calculates `a / b`. Use `try`, `catch`, and `finally` to handle the division-by-zero exception. If `b` is zero, print "Cannot divide by zero". The `finally` block should always print "Operation completed".

Input:

The input contains two integers a and b.

Output:

If b is non-zero, print the integer division result followed by "Operation completed". If b is zero, print "Cannot divide by zero" followed by "Operation completed".

Example:

Input:

20 5

Output:

4

Operation completed

Q5. HashSet – Unique Elements – 5 Marks

Problem Statement:

You are processing a list of product IDs received from multiple transactions. The same product ID may appear multiple times, but the system needs to store each product ID only once.

Write a Java program that reads n product IDs and stores them in a HashSet<Integer>. Print the number of unique product IDs.

Input:

The first line contains an integer n, where $1 \leq n \leq 100$. The second line contains n integers representing product IDs.

Output:

Print the number of unique product IDs.

Example:

Input:

7

101 102 101 103 102 104 101

Output:

4

Q6. TreeSet + Exception Handling – Sorted Unique Numbers – 10Marks

Problem Statement:

You are developing a data-processing application that receives numbers from different sources. The application needs to remove duplicate numbers and display the remaining values in sorted order. The input may also contain invalid values. To keep the program robust, invalid input should be handled using exception handling.

Write a Java program that reads n integer values and stores them in a `TreeSet<Integer>`. The `TreeSet` should automatically remove duplicate values and maintain the elements in ascending order. If an invalid integer is encountered while processing the input, handle the exception and print "Invalid Input".

Input:

The first line contains an integer n , where $1 \leq n \leq 100$. The next n values represent integers.

Output:

For valid input, print the unique numbers in ascending order. If an invalid value is encountered, print "Invalid Input".

Example:

Input:

7

40 10 30 20 40 10 50

Output:

10 20 30 40 50

Q7. [5 Marks - Medium]

You are building a core banking module that must prevent customers from withdrawing more money than is available in their account balance. Whenever such an invalid withdrawal is attempted, a clear custom error should be raised and handled gracefully instead of allowing an invalid transaction to proceed silently.

Problem Statement:

Design a custom exception class `InsufficientBalanceException` that extends `Exception`. Write a program that reads two integers, `balance` and `withdrawalAmount`. If `withdrawalAmount` is greater than `balance`, throw an `InsufficientBalanceException` with the message "Insufficient balance", catch it in the main method, and print the exception's message. Otherwise, print the new balance after the withdrawal.

Input:

Two integers, balance and withdrawalAmount, separated by space.

Output:

Print the new balance after withdrawal, or the exception message "Insufficient balance" if the withdrawal cannot be completed.

Example:

Input:

5000 7000

Output:

Insufficient balance

Q8. [5 Marks - Medium]

You are building a browsing-history feature for a web browser application. The history of visited page IDs is stored in the order they were visited, but the feature must be able to display them in reverse chronological order, with the most recently visited page shown first.

Problem Statement:

Write a program that reads an integer n , followed by n integers representing page IDs, stores them in an `ArrayList<Integer>`, and prints the elements of the list in reverse order, space-separated.

Input:

The first line contains an integer n ($1 \leq n \leq 100$). The second line contains n space-separated integers.

Output:

Print the n integers in reverse order, space-separated.

Example:

Input:

5
10 20 30 40 50

Output:

50 40 30 20 10

Q9. [10 Marks - Hard]

You are building an inventory de-duplication tool for a warehouse management system. Product codes are entered by warehouse staff, but duplicate entries occasionally occur due to manual entry errors. The system must store only unique product codes in sorted order and report how many duplicate entries were detected during entry.

Problem Statement:

Write a program that reads an integer n , followed by n integer product codes. Insert the codes into a `TreeSet<Integer>`. Print each unique product code on its own line in ascending sorted order, followed by a final line printing the number of duplicate entries that were skipped (i.e., n minus the number of unique codes).

Input:

The first line contains an integer n ($1 \leq n \leq 1000$). The second line contains n space-separated integers.

Output:

Print each unique product code on its own line in ascending order, followed by a final line with the count of duplicates skipped.

Example:**Input:**

```
6
102 101 103 101 104 102
```

Output:

```
101
102
103
104
2
```

Q10. [5 Marks - Medium]

You are developing a calculator application that must handle invalid arithmetic operations gracefully, such as division by zero, without crashing the entire application or showing an unhandled error to the user.

Problem Statement:

Write a program that reads two integers a and b , and attempts to compute and print a / b . If b is 0, catch the `ArithmeticException` that is thrown and print "Cannot divide by zero" instead.

Input:

Two integers a and b, separated by space.

Output:

Print the result of a / b , or "Cannot divide by zero" if b is 0.

Example:

Input:

20 4

Output:

5

Q11. [5 Marks - Medium]

You are building a queue-monitoring tool for a customer service center. Customers are added to a waiting list in order of arrival, but the system needs to remove the very first and the very last customer from the list (for example, when the first is served and the last cancels), then display who remains.

Problem Statement:

Write a program that reads an integer n , followed by n integers representing customer IDs, stores them in a `LinkedList<Integer>`, removes the first and last elements from the list, and prints the remaining elements space-separated. Assume $n \geq 2$.

Input:

The first line contains an integer n ($2 \leq n \leq 100$). The second line contains n space-separated integers.

Output:

Print the remaining customer IDs after removing the first and last, space-separated. If no elements remain, print an empty line.

Example:

Input:

5
11 22 33 44 55

Output:

22 33 44

Q12. [10 Marks - Hard]

You are building the core lending module of a library management system. Each book has a title and a number of available copies. When a member requests a book, the system must issue a copy if available, but must raise a clear, custom error when the requested book cannot be issued because it does not exist in the catalog or has zero copies remaining.

Problem Statement:

Design a class Book with fields title (String) and copies (int). Design a custom exception class BookNotAvailableException that extends Exception. Maintain the catalog as an ArrayList<Book>. Write a program that reads an integer n (number of books), followed by n lines each containing a book title and its number of copies, then reads a title requested for issue. Search the catalog for the given title (exact match): if found and copies > 0, decrement its copies by 1 and print "Issued: " followed by the title; if the title is not found, or found with 0 copies, throw a BookNotAvailableException with the message "Book not available: " followed by the title, catch it in main, and print the exception's message.

Input:

The first line contains an integer n ($1 \leq n \leq 50$). The next n lines each contain a book title (a single word, no spaces) and an integer copies count, separated by a space. The last line contains the title of the book requested for issue.

Output:

Print "Issued: <title>" if the book is successfully issued, otherwise print the custom exception's message.

Example:

Input:

```
3
JavaBasics 2
DataStructures 0
Algorithms 1
JavaBasics
```

Output:

```
Issued: JavaBasics
```

Q13. [5 Marks - Medium]

You are building a simple grading utility for a school application. Marks entered by a teacher for a class need to be stored conveniently and their average computed to report the overall class performance.

Problem Statement:

Write a program that reads an integer n , followed by n integers representing marks, stores them in an `ArrayList<Integer>` (relying on Java's autoboxing to convert each `int` into an `Integer` object), and prints their average rounded to 2 decimal places.

Input:

The first line contains an integer n ($1 \leq n \leq 100$). The second line contains n space-separated integers.

Output:

Print the average of the n marks, rounded to 2 decimal places.

Example:

Input:

4
80 90 70 60

Output:

75.00

Q14. [5 Marks - Medium]

You are building an attendance-verification tool for an event-management system. Attendee IDs are scanned as people enter, but occasionally the same ID gets scanned twice by mistake. The system needs to report how many distinct attendees actually entered the event.

Problem Statement:

Write a program that reads an integer n , followed by n integers representing scanned attendee IDs (which may contain duplicates), stores them in a `HashSet<Integer>`, and prints the count of unique attendee IDs.

Input:

The first line contains an integer n ($1 \leq n \leq 1000$). The second line contains n space-separated integers.

Output:

Print a single integer, the number of unique attendee IDs.

Example:**Input:**

```
6
101 102 101 103 102 104
```

Output:

```
4
```

Q15. [10 Marks - Hard]

You are building the stock-intake module of an inventory management system. Warehouse staff enter quantities received for various items, but sometimes a negative quantity is mistakenly entered due to a data-entry error. The system must reject such invalid entries with a clear custom error while still successfully processing all valid entries and reporting the total stock received.

Problem Statement:

Design a custom exception class `InvalidQuantityException` that extends `Exception`. Write a program that reads an integer n , followed by n integers representing quantities received. For each quantity, if it is negative, throw an `InvalidQuantityException` with the message "Invalid quantity: " followed by the value, catch it immediately, print the message, and skip that value; otherwise, add it to an `ArrayList<Integer>` of valid quantities. After processing all n quantities, print the sum of all valid quantities.

Input:

The first line contains an integer n ($1 \leq n \leq 100$). The second line contains n space-separated integers, which may include negative values.

Output:

For each negative quantity, print "Invalid quantity: <value>". After processing all values, print the total sum of the valid (non-negative) quantities.

Example:**Input:**

```
5
10 -5 20 -3 15
```

Output:

Invalid quantity: -5

Invalid quantity: -3

45